TryHackMe — Pre-Security Learning Path

Packets & Frames

Learning Notes — TCP, UDP, Encapsulation & Ports

> ***"Data does not travel across a network as one big stream — it is broken into small pieces called packets, sent independently, and reassembled at the destination."***

Packets vs Frames | TCP & Handshake | UDP Protocol | Ports & Services

1. What Are Packets and Frames?

Packets and frames are both small units of data used in networking, but they exist at different layers of the OSI model and carry different types of addressing information. Understanding the distinction is fundamental to understanding how data actually travels across a network.

	Packet	Frame
OSI Layer	Layer 3 — Network	Layer 2 — Data Link
Addressing	Contains IP addresses (source and destination)	Contains MAC addresses — no IP addresses
Scope	Travels across multiple networks (inter-network)	Travels within a single local network segment
Analogy	The outer envelope you post to a remote address	The inner envelope that survives inside the outer one
Protocol	IP (Internet Protocol)	Ethernet (most common)
What it carries	The IP header + the TCP/UDP segment	The IP packet wrapped in MAC header + CRC trailer

The Envelope Analogy

Think of a packet like a letter inside an addressed outer envelope — it has a destination IP address for routing across the Internet. A frame is like putting that outer envelope inside a second inner envelope for delivery on your local street (LAN). Once the outer layer is stripped away, the inner frame still exists and carries the remaining data to the next local device.

1.1 Why Small Pieces?

Large files are never sent as one continuous stream across a network. They are broken into many small packets before transmission. This design has important practical benefits:

- Reduces bottlenecking — multiple smaller packets can take different routes and be transmitted in parallel.
- Enables error recovery — if one packet is lost or corrupted, only that packet needs to be retransmitted, not the entire file.
- Allows fair sharing — many devices can share the same network simultaneously without any single device monopolising the bandwidth.
- Packets are reassembled in the correct order at the destination using sequence numbers.

1.2 Packet Anatomy — Header and Data

Every packet (and frame) is made up of two core sections:

Section	Contents and Purpose
Header	Logistical and technical information: source/destination addresses, TTL, checksum, sequence numbers, flags. Gets the data to the right place safely.
Data (Payload)	The actual content being transmitted — bytes of a file, an HTTP request, a video chunk, an email body. This is what the application actually wants to receive.

Common Header Fields

TTL (Time To Live) — limits how many hops a packet can travel before being discarded. Checksum — a mathematical value used to detect corruption during transit. Source Address — where the data came from. Destination Address — where the data is going. These fields appear in both packets (IP addresses) and frames (MAC addresses).

2. TCP — Transmission Control Protocol

TCP is a connection-oriented, reliable transport protocol. Before any data is sent, TCP requires both devices to establish a connection through the Three-Way Handshake. This guarantees that data will arrive, arrive in order, and arrive uncorrupted.

2.1 TCP Advantages and Disadvantages

Advantages of TCP	Disadvantages of TCP
Guarantees delivery — every packet is acknowledged	Slower than UDP due to overhead from acknowledgements
Ensures data integrity via checksum verification	Requires a reliable connection to function correctly
Synchronises devices and prevents data flooding	If one packet is lost, the entire chunk cannot be used until retransmitted
Reassembles data in the correct sequence order	A slow connection can bottleneck the other device
Error checking built into every packet exchange	Reserves system resources for the entire duration of the connection

2.2 TCP Packet Header Fields

TCP packets contain specific header fields that together ensure reliable, ordered, and error-checked delivery. Each field serves a defined purpose:

Header Field	Description	Purpose
Source Port	Port number opened by the sender — chosen randomly from 0–65535 ports not already in use.	Identifies the sending application on the source host.
Destination Port	Port number on the remote host where the application/service is running (e.g. port 80 for HTTP).	Routes the packet to the correct application at the destination.
Source IP Address	IP address of the device sending the packet.	Identifies the origin so the destination knows where to reply.
Destination IP	IP address of the device the packet is being sent to.	Ensures the packet reaches the correct recipient device.
Sequence Number	A randomly assigned number used to order the data and reconstruct packets correctly.	Allows the receiver to reassemble out-of-order packets into the correct sequence.

Acknowledgement	When data has been sent, this number is the next expected sequence number (current ISN + 1).	Confirms receipt and tells the sender which byte is expected next.
Checksum	A mathematical calculation over the packet contents — sender computes it; receiver recalculates and compares.	Data integrity — if checksums differ, the packet is corrupt and discarded.
Data	The actual payload being transmitted — bytes of a file, HTTP content, etc.	Carries the application-layer information that needs to be delivered.
Flags	Control bits: SYN, ACK, FIN, RST, PSH, URG — each triggers specific handling behaviour.	Controls connection setup, maintenance, termination, and data prioritisation.

3. TCP Flags

TCP flags are control bits within the TCP header that determine how a packet should be handled. They govern every stage of a TCP connection — opening, transferring data, and closing. Understanding flags is essential for network analysis and security work.

SYN	SYN/ACK	ACK	DATA	FIN	RST	PSH	URG
Open connection request	Server acknowledges	Received OK	Payload transfer	Close connection	Force reset	Deliver now	Priority data

Flag	Full Name	When It Is Used
SYN	Synchronise	Sent by the client to initiate a connection. Includes the client's Initial Sequence Number (ISN). Always the first message in the Three-Way Handshake.
SYN/ACK	Synchronise-Ack	Sent by the server in response to the client's SYN. Acknowledges the client's ISN and provides the server's own ISN.
ACK	Acknowledgement	Used by either device to confirm that a series of packets or a specific message was successfully received. Used throughout the entire connection lifecycle.
DATA	Data Transfer	Not technically a standard flag name — refers to the packet type once data payload is being transmitted after the handshake is complete.
FIN	Finish	Sent by a device that wants to close the connection cleanly. The other device must also send FIN+ACK to complete the four-step connection teardown.
RST	Reset	Forcefully terminates a connection — used when something has gone wrong, such as a connection attempt to a closed port. Also used in port scanning.
PSH	Push	Instructs the receiver to deliver buffered data to the application immediately, rather than waiting for the buffer to fill. Used for interactive applications.
URG	Urgent	Marks data as urgent — the receiver should prioritise processing it ahead of other data in the buffer. Rarely used in practice.

4. The TCP Three-Way Handshake

Before TCP can transmit any data, both devices must agree on connection parameters — specifically their Initial Sequence Numbers (ISNs). This agreement happens through the Three-Way Handshake: a three-message exchange that establishes a synchronised, reliable channel.

4.1 Opening a Connection — The Three-Way Handshake

Step	Message	From → To	What It Means
1	**SYN**	Client → Server	Client sends its ISN (e.g. 0) and requests to synchronise. "Hi server, here is my sequence number (0) — can we connect?"
2	**SYN/ACK**	Server → Client	Server acknowledges client's ISN (0) and sends its own ISN (e.g. 5,000). "I acknowledge your number (0). Here is mine (5,000)."
3	**ACK**	Client → Server	Client acknowledges server's ISN (5,000+1 = 5,001). Connection established. Data transfer can now begin.

Initial Sequence Numbers

Both devices generate a random starting number (ISN) before the handshake. During the handshake, each device learns the other's ISN. All future sequence numbers are based on incrementing from these starting points. This is how TCP tracks which bytes have been received and which need retransmission.

4.2 Transferring Data

Once the handshake is complete, data is transmitted as a series of DATA packets. Each DATA packet is acknowledged by the receiver with an ACK. The sequence and acknowledgement numbers increment with each exchange, keeping both sides in sync.

Step	Message	From → To	What It Means
4	**DATA**	Client → Server	Client sends data (e.g. "Cheesecake is on sale!") inside a TCP DATA packet.
5	**ACK**	Server → Client	Server confirms receipt. "I received your data correctly."

4.3 Closing a Connection — Four-Step Teardown

When data transfer is complete, TCP closes the connection cleanly using a four-step exchange. This ensures both sides have finished sending before freeing up system resources.

Step	Message	From → To	What It Means
6	FIN	Client → Server	Client signals it is done sending data. "I have no more data to send — let's close."
7	ACK	Server → Client	Server acknowledges the client's FIN. "Got it. I'm also ready to close."
8	FIN	Server → Client	Server sends its own FIN to confirm it is also done. "I'm done too — closing my side."
9	ACK	Client → Server	Client sends final ACK. Connection is now fully closed. Resources freed on both sides.

RST — The Emergency Exit

If something goes wrong mid-connection — e.g. a device crashes, a connection attempt is made to a closed port, or a firewall terminates the session — a RST (Reset) packet is sent. This forcefully terminates the connection without the clean FIN-ACK exchange. RST packets are a common indicator of abnormal network behaviour and are visible in Wireshark during port scanning.

4.4 Hands-On Lab — Re-assembling the Handshake (Alice & Bob)

The TryHackMe room includes an interactive lab where you help Alice and Bob communicate by placing the TCP handshake messages in the correct order. The lab simulates a full TCP session from connection to closure.

Order	Message	Sender	Lab Content
1st	SYN	Alice (Client)	Alice initiates the connection with her ISN (0).
2nd	SYN/ACK	Bob (Server)	Bob acknowledges Alice's ISN and sends his ISN (5,000).
3rd	ACK	Alice (Client)	Alice confirms Bob's ISN (5,001). Connection established.
4th	DATA	Alice → Bob	Alice sends: "Cheesecake is on sale!"
5th	ACK	Bob → Alice	Bob confirms receipt of Alice's data.
6th	FIN/ACK	Alice → Bob	Alice signals she has no more data and wants to close.
7th	FIN/ACK	Bob → Alice	Bob confirms and signals he also wants to close.

8th	ACK	Alice → Bob	Alice sends the final ACK. Connection fully closed.

Flag Captured

THM{TCP_CHATTER} — Successfully completing the TCP conversation in the correct order confirms understanding of the full TCP session lifecycle, from SYN through to the final ACK closure.

5. UDP — User Datagram Protocol

UDP (User Datagram Protocol) is the alternative to TCP at the Transport Layer. Unlike TCP, UDP is stateless — it sends data without establishing a connection first, without acknowledgements, and without guaranteeing delivery or order. What it lacks in reliability, it makes up for in speed.

5.1 UDP Advantages and Disadvantages

Advantages of UDP	Disadvantages of UDP
Much faster than TCP — no handshake or acknowledgement overhead	No guarantee of delivery — packets may be silently dropped
No reserved connection — lower resource usage on both devices	No guarantee of order — packets may arrive out of sequence
Application software can control packet send rate directly	No error checking or retransmission — data loss is permanent
Ideal where some data loss is acceptable (streaming, gaming)	No synchronisation — no protection against packet flooding
Lower latency — critical for real-time applications	Unsuitable for applications requiring perfect data integrity (file transfer)

5.2 UDP Packet Headers

UDP packets are significantly simpler than TCP packets — they have far fewer header fields, which contributes to their speed advantage:

Header Field	Purpose
TTL (Time To Live)	Limits how many hops the datagram can travel before being discarded — prevents endless looping.
Source Address	IP address of the device sending the datagram.
Destination Address	IP address of the device the datagram is destined for.
Source Port	Port used by the sending application — randomly chosen from available ports.
Destination Port	Port of the target application or service on the receiving device.
Data	The payload — the actual content being sent (video chunk, audio sample, DNS query, etc.).

No Checksum, No Sequence, No Acknowledgement

Compare UDP's six header fields to TCP's nine — UDP deliberately omits checksum integrity enforcement, sequence numbers, and acknowledgement numbers. There is no connection setup and no teardown. Data is simply fired and forgotten. This is exactly what makes UDP ideal for live streaming, where a missing video frame is better tolerated than pausing to request a retransmission.

5.3 UDP Connection Flow

Because UDP is stateless, there is no handshake. The sender simply transmits datagrams and the receiver either gets them or does not. No acknowledgements are sent:

Step	Action	What Happens
1	**Send datagram**	Sender transmits a UDP datagram directly to the destination IP and port — no setup required.
2	**Receive or drop**	Receiver either processes the datagram if the port is open, or drops it silently.
3	**No acknowledgement**	Sender receives no confirmation. It does not know if the datagram was received.
4	**Next datagram**	Sender immediately transmits the next datagram — no waiting for acknowledgement.

5.4 TCP vs. UDP — When to Use Which

Use Case	Protocol	Why
File transfer (FTP, SFTP)	**TCP**	Data integrity is critical — a missing byte corrupts the entire file.
Web browsing (HTTP/HTTPS)	**TCP**	Web pages must arrive complete and in order for correct rendering.
Email (SMTP, IMAP, POP3)	**TCP**	Messages must arrive whole and uncorrupted.
Video streaming (YouTube, Netflix)	**UDP**	A dropped frame is invisible; pausing to retransmit ruins the experience.
Voice/video calls (VoIP, Zoom)	**UDP**	Low latency is critical; brief audio/video glitches are acceptable.

Online gaming	**UDP**	Real-time position updates need speed — outdated data is worthless anyway.
DNS queries	**UDP**	Short, simple request-response — low overhead matters; TCP fallback for large responses.
DHCP	**UDP**	Broadcast-based address assignment — no established connection exists yet.

6. Ports

A port is a numerical identifier (0–65535) used to direct network traffic to the correct application or service on a device. When a packet arrives at a device, the IP address gets it to the right machine — and the port number gets it to the right application on that machine.

The Port Analogy

Think of an IP address as the street address of a building, and a port as the door number within that building. Both are needed: the IP address routes your data to the right device, and the port routes it to the right application (process) running on that device.

6.1 Port Ranges

Range	Category	Details
0 – 1023	Well-Known Ports	Reserved for standard services by IANA. Any application using these requires administrator/root privileges. Most important for networking and security professionals to memorise.
1024 – 49151	Registered Ports	Assigned to specific software vendors and applications. Less critical to memorise but useful to recognise (e.g. 3306 = MySQL, 5432 = PostgreSQL, 8080 = HTTP alternate).
49152 – 65535	Dynamic / Ephemeral	Randomly assigned as source ports for outgoing connections. The source port in a TCP/UDP header is chosen from this range (or the full 0–65535 range minus in-use ports).

6.2 Well-Known Ports — Essential Reference

Port	Protocol	Transport	Service / Usage
21	FTP	TCP	File Transfer Protocol — transfers files between client and server.
22	SSH	TCP	Secure Shell — encrypted remote command-line access to a system.
23	Telnet	TCP	Unencrypted remote terminal access — legacy, insecure. Do not use.
25	SMTP	TCP	Simple Mail Transfer Protocol — sending email between servers.
53	DNS	UDP/TCP	Domain Name System — resolves domain names to IP addresses.
80	HTTP	TCP	HyperText Transfer Protocol — unencrypted web traffic.
110	POP3	TCP	Post Office Protocol 3 — downloading email from a mail server.
143	IMAP	TCP	Internet Message Access Protocol — accessing email on a server.
443	HTTPS	TCP	HTTP Secure — encrypted web traffic using TLS/SSL.
3389	RDP	TCP/UDP	Remote Desktop Protocol — graphical remote access to Windows systems.

6.3 Hands-On Lab — Connecting to a Port (Netcat)

The room ends with a practical lab where you connect to a specific IP address and port using the nc (netcat) command — a fundamental tool for network connections and security testing.

```
# Connect to IP 8.8.8.8 on port 1234 using netcat
nc 8.8.8.8 1234

# The simulator constructs this command from the IP and port you enter.
# Netcat establishes a TCP connection to the target IP:port.
# If the port is open and listening, the connection succeeds
# and any data sent by the remote service is displayed.
```

Flag Captured

THM{YOU_CONNECTED_TO_A_PORT} — Connecting to IP 8.8.8.8 on port 1234 in the lab simulator reveals this flag, demonstrating that port-level connections are the foundation of all network communication — and all network-based attacks.

Why Netcat Matters for Security

Netcat (nc) is often called the 'Swiss Army knife' of networking. It can open TCP/UDP connections, listen on ports, transfer files, and even create reverse shells. Penetration testers and attackers use it constantly. Understanding how to connect to a port is the first step towards understanding port scanning (Nmap), banner grabbing, and service exploitation.

7. Security Implications

Packet Analysis Is the Foundation of Network Security

Every tool used in network security — Wireshark, Nmap, Snort, Suricata — works by capturing, parsing, and analysing packets and frames. Understanding what a TCP SYN packet looks like, what a normal three-way handshake sequence is, and what an anomalous RST storm indicates — these are the core skills that separate guessing from genuine analysis.

TCP Flags Reveal Attack Intent

Nmap's stealth SYN scan sends SYN packets but never completes the handshake (responds with RST instead of ACK). Xmas scans set PSH+URG+FIN flags simultaneously — which no normal application ever does. NULL scans send no flags at all. Recognising abnormal flag combinations is how defenders spot reconnaissance.

Port Numbers Define the Attack Surface

Every open port is a potential entry point. Port scanning (Nmap) maps the attack surface by probing ports 0–65535 for responses. A port responding to SYN means a service is running — and every service has known vulnerabilities. Closing unnecessary ports is one of the most effective hardening measures available.

Sequence Numbers Enable Both Reliability and Attack

TCP's sequence numbers ensure data arrives in order and detect missing packets. But in TCP session hijacking, an attacker predicts the next sequence number to inject malicious data into an existing connection — exploiting the very mechanism that makes TCP reliable. Modern operating systems use randomised ISNs specifically to prevent this.

UDP's Lack of Verification Is Exploitable

Because UDP has no connection state, attackers can forge (spoof) the source IP in a UDP packet — the receiver has no way to verify the sender's identity. UDP amplification attacks (DNS amplification, NTP amplification) exploit this: send a small spoofed UDP request to a service that responds with a large reply, directing the flood at the victim's IP.

8. Key Takeaways & Next Steps

8.1 Core Concepts Glossary

Term	Definition
Packet	A unit of data at Layer 3 (Network) containing IP source/destination addresses.
Frame	A unit of data at Layer 2 (Data Link) containing MAC addresses — no IP addresses.
Encapsulation	The process of wrapping data with header information at each OSI layer as it travels down the stack.
De-encapsulation	The reverse — stripping headers at each layer as data travels up the stack at the receiver.
TCP	Transmission Control Protocol — connection-oriented, reliable, ordered, error-checked transport.
UDP	User Datagram Protocol — stateless, fast, no guaranteed delivery, minimal overhead.
Three-Way Handshake	SYN → SYN/ACK → ACK — the TCP connection establishment sequence.
ISN	Initial Sequence Number — a random starting number agreed during the TCP handshake.
Checksum	A mathematical integrity check in the TCP header — detects corruption in transit.
SYN / ACK / FIN	TCP control flags governing connection open (SYN), acknowledgement (ACK), and close (FIN).
RST	TCP Reset flag — forcefully terminates a connection; seen in port scanning and errors.
Port	A numerical identifier (0–65535) routing traffic to the correct application on a device.
Well-Known Ports	Ports 0–1023 — reserved for standard services (HTTP=80, HTTPS=443, SSH=22, DNS=53).
Netcat (nc)	Command-line tool for making TCP/UDP connections — fundamental networking and security utility.

8.2 Next Steps on TryHackMe

- **Immediate:** Continue with the Extending Your Network room — VLANs, subnetting, firewalls, and network design.
- **Applied:** Complete the Wireshark: The Basics room — capture and analyse real TCP/UDP packets and frames.
- **Security:** Study the Nmap room — port scanning uses TCP flags (SYN, RST, ACK) directly.
- **Security:** Take the Network Security module — real-world attacks against the layers covered here.
- **Practice:** Practice reading packet captures (PCAPs) in Wireshark — apply everything from this room visually.

The Bigger Picture

Packets and frames are the atoms of networking. Every concept that follows — routing, firewalls, intrusion detection, VPNs, TLS — is built on top of the fundamentals covered in this room. Mastering how data moves across a network at the packet level makes every other security topic easier to understand and apply.

Source: TryHackMe — Packets & Frames Room | tryhackme.com/room/packetsframes